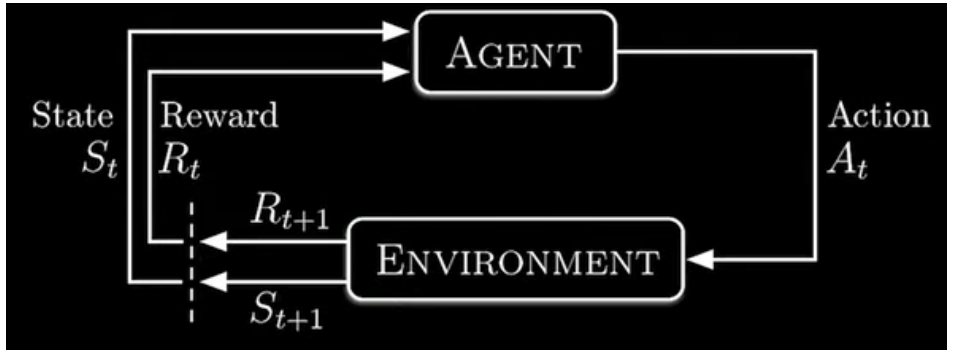
CSCN8020 – Week 6

Learning Activities

Recap: Finite Markov Decision Process

The process repeats itself until some terminal state is reached, marking the end of an episode (e).



$s \in S$

S^+ if we include terminal states

$a \in A(s)$

$r \in R \in \mathbb{R}$

$p(s', r|s, a)$

Probability of the next state and reward given the current state and action.

$\pi(a|s)$

The policy is a state-dependent distribution over a set of actions.

$$e_k = S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, \ldots, R_T, S_t$$

If we run the policy over the MDP, we get a **Trajectory** over an episode.

We want the gamma discounted sum of future rewards to be high when averaged over many trajectories.

$$G_t = \sum_{k=t+1}^{T} \gamma^{k-t-1} R_k$$

Goal $max_\pi \, \mathbb{E}[G_t]$

To achieve that goal, we consider state and action value functions.

$$v_\pi(s) = \mathbb{E}[G_t | S_t = s]$$

$$q_\pi(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a]$$

We find an optimal policy if we can find the optimum of these functions.

$$v_*(s) = \max_\pi v_\pi(s)$$

$$q_*(s, a) = \max_\pi q_\pi(s, a)$$

Recap: Q-Learning

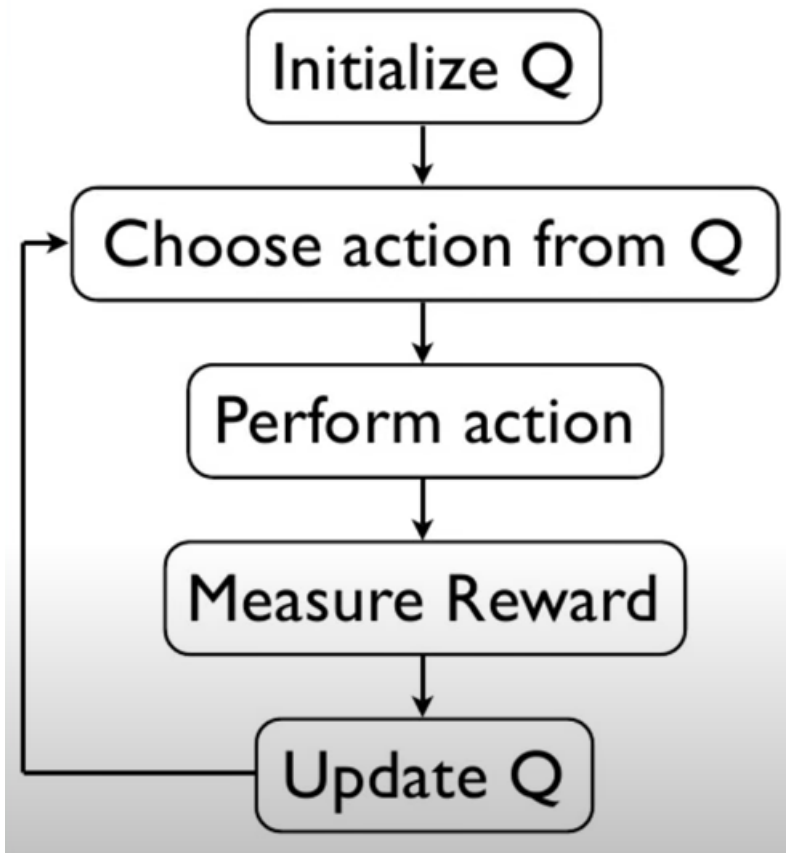
In RL, we want to obtain a function Q(s, a) that predicts the best action a in a state s to maximize the cumulative reward. This function can be estimated using Q-learning, which iteratively updates Q(s, a) using the Bellman Equation:

We can use TD(0) to learn value <u>and</u> Q functions.

- Off-policy TD control algorithm
- $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)\right]$
- All pairs (state, action) need to continue being visited and updated
- Off-policy: Q-values are updated based on the maximum estimated future return, even if the action taken in that state was not the one recommended by the current policy. This enables Q-Learning to learn a policy that may differ from the one it uses for exploration.

Learning the Q function is called Q-learning.

Recap: Q-Learning



State with → R = 1
100% of the time

State with → R = -1
90% of the time
And → R = 20
10% of the time

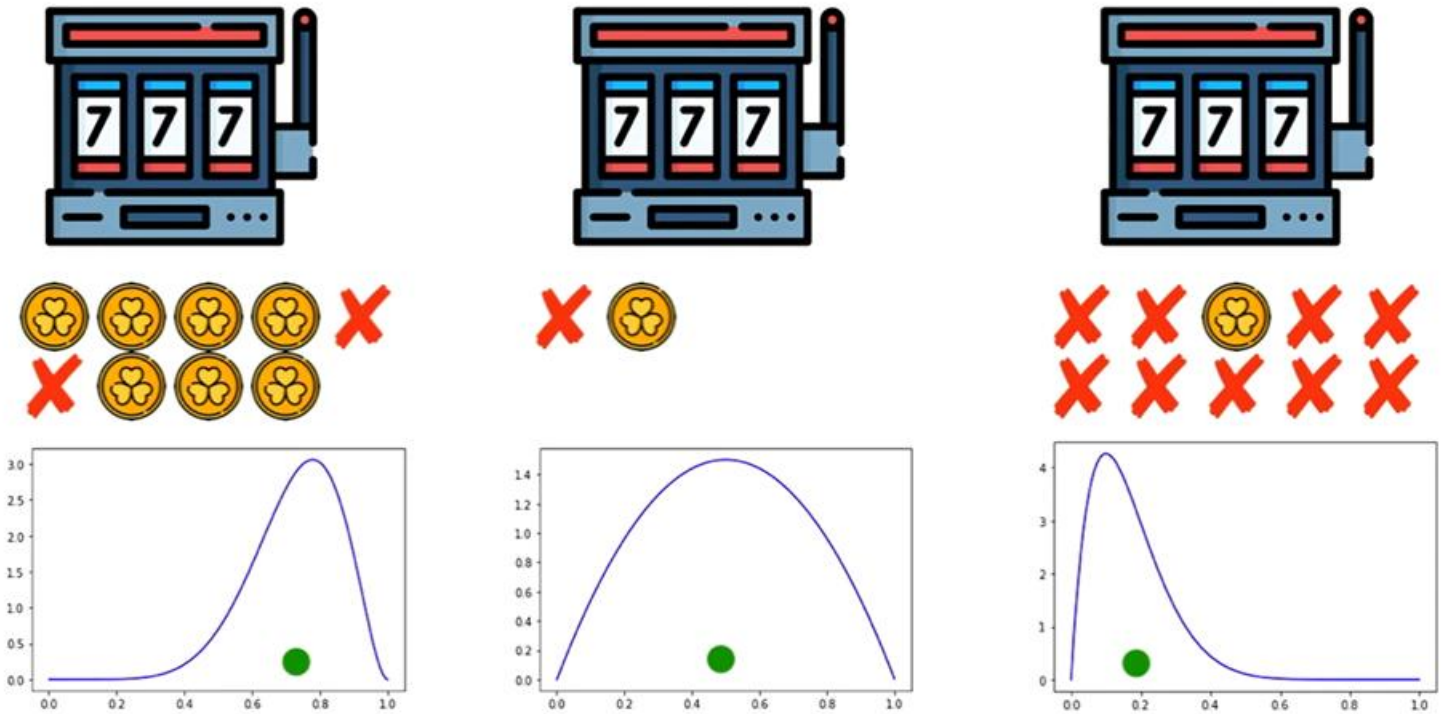
A greedy function will always (100%) go to the state with the highest reward.

A random exploration function sometimes goes to the right (10%).

Thompson Sampling

Thompson Sampling for Multi-armed Bandit (MAB)

Watch the video on the class notes, slide 11/15



Successful:
Very likely to get picked

Unexplored:
Likely to get picked

Unsuccessful:
Unlikely to get picked

1

Source: https://www.youtube.com/watch?v=nkyDGGQ5h60

Thompson Sampling & RL: Robot Maze

Step 1: Define the Maze Environment

We consider a **5×5 grid maze** where a robot must navigate from the **Start (S)** to the **Goal (G)**. There are **two main paths** available:

1. **Path A (Riskier, Shorter)**
   - **3 steps** to goal.
   - Reward = 1 with probability $P_A = 0.7$.
   - Can sometimes lead to a **trap (T)**, requiring backtracking.
2. **Path B (Safer, Longer)**
   - **5 steps** to goal.
   - Reward = 1 with probability $P_B = 0.9$.
   - No traps but takes longer.

Graphical Representation of the Maze

nginx Copy Edit

```nginx
S → A1 → A2 → G
↓    ↓    ↓
B1 → B2 → B3 → B4 → B5 → G
```

- **S (Start)**
- **A1, A2 (Riskier path, shorter)**
- **B1, B2, B3, B4, B5 (Safer path, longer)**
- **G (Goal, reward = 1 if reached successfully)**

Thompson Sampling & RL: Robot Maze

Step 2: Model the Problem as a Multi-Armed Bandit

Each **path** represents a **bandit arm** with an **unknown probability of success**:

- **Path A:** Success probability $\theta_A \sim \text{Beta}(1, 1)$.
- **Path B:** Success probability $\theta_B \sim \text{Beta}(1, 1)$.

Thompson Sampling & RL: Robot Maze

Step 3: Thompson Sampling for Path Selection

1. **Initialize Beliefs:**
   - Start with uniform priors:
     - $\text{Beta}(1, 1)$ for both paths.
2. **First Decision (Episode 1):**
   - Sample $\theta_A \sim \text{Beta}(1, 1)$, e.g., **0.85**.
   - Sample $\theta_B \sim \text{Beta}(1, 1)$, e.g., **0.60**.
   - Since $\theta_A > \theta_B$, **robot chooses Path A**.
3. **Observe Outcome:**
   - With probability 0.7, Path A is successful.
   - If successful → update $\alpha_A \leftarrow \alpha_A + 1$.
   - If failed → update $\beta_A \leftarrow \beta_A + 1$ and retry.
4. **Subsequent Episodes:**
   - The robot repeats the process, gradually **learning the real probabilities**.
   - If it encounters more failures in Path A, the Beta distribution updates, and Thompson Sampling starts favoring Path B.

Thompson Sampling & RL: Robot Maze

Step 4: Updating Probabilities Over Time

Example of Belief Updates Over 5 Trials

Episode	Chosen Path	Sampled θ_A	Sampled θ_B	Outcome	Updated Beta
1	A	0.85	0.60	Fail	$\text{Beta}(1, 2)$
2	B	0.72	0.82	Success	$\text{Beta}(2, 1)$
3	B	0.65	0.91	Success	$\text{Beta}(3, 1)$
4	A	0.92	0.80	Success	$\text{Beta}(2, 2)$
5	A	0.80	0.75	Success	$\text{Beta}(3, 2)$

Step 5: Convergence to the Best Path

- Initially, the robot **explores both paths.**
- As it accumulates data, it **adapts and exploits the better path.**
- Over multiple episodes, it converges to **Path B**, since $P_B = 0.9$ is better in the long run.

Thompson Sampling & RL: Robot Maze

Final Outcome

- Initially, the robot tries Path A due to **random exploration.**
- After experiencing more failures in A, it **shifts towards Path B.**
- **After sufficient trials, the robot reliably selects the best path (B) to reach G.**

Thompson Sampling & RL: Robot Maze

Python Simulation

MAB-Simulation	2025-02-20 7:19 PM	Jupyter Source File	3 KB

python

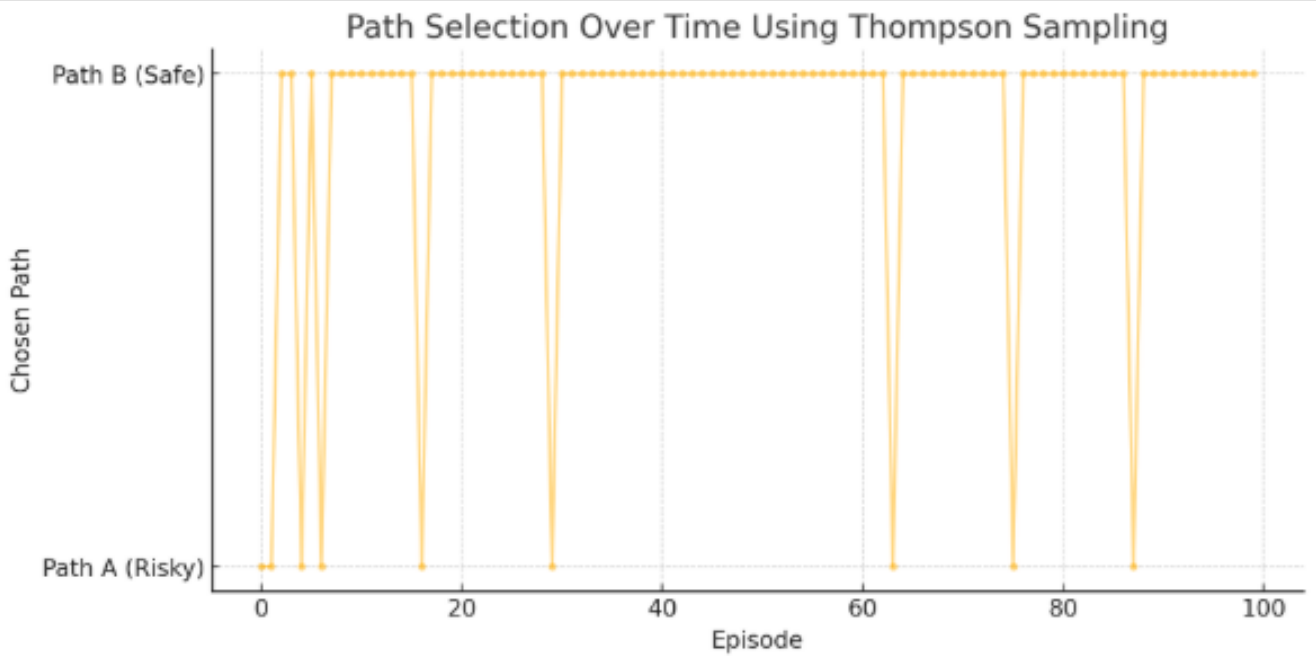
Always show details Copy

```python
import numpy as np
import matplotlib.pyplot as plt

# Initialize priors for Path A and Path B (Beta distributions)
alpha_A, beta_A = 1, 1  # Path A
alpha_B, beta_B = 1, 1  # Path B

# True success probabilities (unknown to the agent)
true_P_A = 0.7  # Path A (riskier, shorter)
true_P_B = 0.9  # Path B (safer, longer)

# Simulation parameters
num_episodes = 100  # Number of trials
chosen_paths = []  # Store chosen paths for visualization
success_counts = {"A": 0, "B": 0}  # Track successes
```

Result

```
({'A': 6, 'B': 81}, 7, 4, 82, 11)
```

Path Selection Over Time Using Thompson Sampling



Thompson Sampling & RL: Robot Maze

Python Simulation

Here are the results of the Thompson Sampling simulation:

- **Final Success Counts:**
  - Path A (Risky, Shorter): **6 successes**
  - Path B (Safe, Longer): **81 successes**
- **Final Beta Distributions (Beliefs):**
  - Path A: $\text{Beta}(7, 4)$ (indicating lower confidence due to fewer successes)
  - Path B: $\text{Beta}(82, 11)$ (indicating strong confidence in higher success probability)

Observations from the Plot:

- Initially, the robot **explores both paths.**
- Over time, as **Path B proves to be more reliable**, the agent **gradually shifts towards choosing Path B** more often.
- This demonstrates how **Thompson Sampling learns the best path without prior knowledge.**

Exercise #1 – Thompson Sampling and MAB

1. Clone and run the Multi-Armed Bandit Workshop (https://github.com/ProfEspinosaAIML/MultiArmedBandit_Workshop.git)

2. Generate the CSV files from the two rounds: Stationary Casino and Non-Stationary Casino Competition

3. Understand & visualize the exploration–exploitation trade-off, including why ε matters, and why constant-α helps in non-stationary settings.

4. Create your own notebook, incorporating the reflections and visualizations from both competitions.

5. Send the instructor an email with the link to your GitHub repository, which contains the exercise, no later than 6 pm on Friday, October 31.

Reflection #1 – The Final Project

1. Work with your team.

2. Prepare a short (1-2 minute) pitch with your term project idea

3. The instructor will randomly choose one team member to present the idea on the whiteboard.
   - → The key idea (short sentence)
   - → The RL strategy

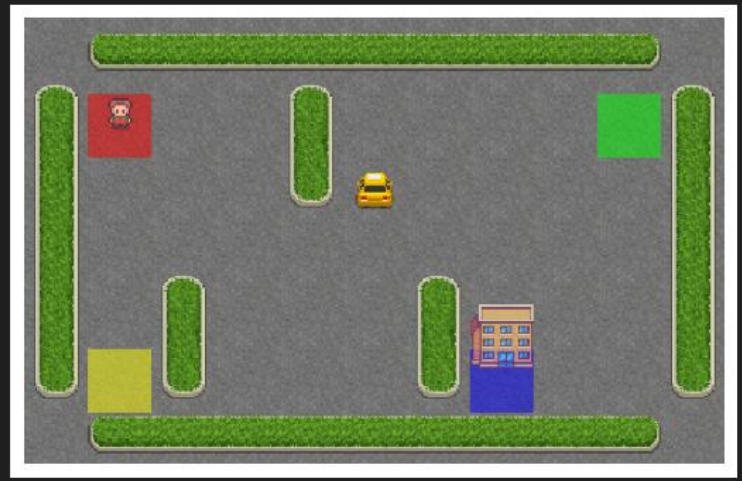
Code Review #1 – Gym/Taxi graphical simulation (localhost)

Gym-Taxi-Environment-Setup.ipynb > Gym - Taxi Environment > This code doesn't run well on Google Colab

Generate | Code | Markdown | Run All | Clear All Outputs | Outline | ... | Select Kernel

```python
sleep(3)

num_steps = 10
for s in range(num_steps+1):

    #clear_output(wait=True)
    print(f"step: {s} out of {num_steps}")

    # sample a random action from the list of available actions
    action = env.action_space.sample()

    # perform this action on the environment
    obs, reward, done, truncated, info = env.step(action)

    # render the frame
    frame = env.render()
    plt.imshow(frame)
    plt.axis('off')
    display(plt.gcf())
    clear_output(wait=True)

    sleep(2)

# end this instance of the taxi environment
env.close()
```

[16] Python



Name	Date modified	Type	Size
.venv	2025-03-04 2:56 PM	File folder	
Gym-Taxi-Environment-Setup	2025-03-05 8:08 PM	Jupyter Source File	386 KB
MAB-Simulation	2025-02-21 6:06 AM	Jupyter Source File	3 KB
openai_gym_taxi_tutorial	2025-03-05 7:30 PM	Jupyter Source File	38 KB

Code Review #2 – Taxi simulation (Google Colab display)

1

Open AI Tutorial:

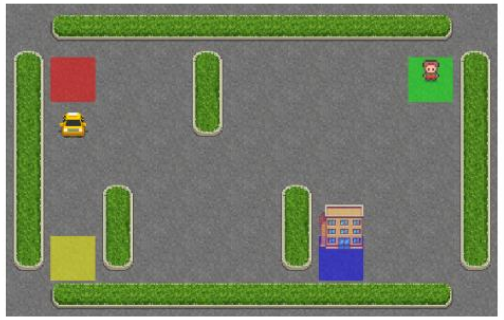
https://www.gocoder.one/blog/rl-tutorial-with-openai-gym/

Name	Date modified	Type	Size
.venv	2025-03-04 2:56 PM	File folder	
Gym-Taxi-Environment-Setup	2025-03-05 8:08 PM	Jupyter Source File	386 KB
MAB-Simulation	2025-02-21 6:06 AM	Jupyter Source File	3 KB
openai_gym_taxi_tutorial	2025-03-05 7:30 PM	Jupyter Source File	38 KB

https://colab.research.google.com/drive/1ZHjAUMIKkz94tjR4RTadjF3rx56DInl8#scrollTo=DikE7qb16c22

Commands + Code + Text

```
try:
    obs, reward, done, truncated, info = env.step(action)  # For Gym versions >= 0.22
except ValueError:
    obs, reward, done, info = env.step(action)  # For older Gym versions
    truncated = False  # Assume episode was not truncated if the value is not returned

# render the frame
clear_output(wait=True) # Call clear_output directly instead of as an attribute of display
plt.figure(figsize=(6, 6))  # Adjust figure size as needed
# Access the correct frame from the rendered output:
rendered_frame = env.render()
# Check if rendered_frame is a list and get the first element if it is:
if isinstance(rendered_frame, list):
    rendered_frame = rendered_frame[0]
# Now you can safely access ndim:
if rendered_frame.ndim == 4:  # Check if the output has an extra dimension
    rendered_frame = rendered_frame[0]  # Select the first frame if it's a sequence

plt.imshow(rendered_frame) # Display the selected frame
plt.axis('off')
plt.show()

sleep(2)

# end this instance of the taxi environment
env.close()
```



peError ValueError AttributeError TypeError AttributeError

Please explain the error:

```
AttributeError: 'list' object has no attribute 'ndim'
```

Hypothesis

The error message `AttributeError: 'list' object has no attribute 'ndim'` indicates that the `rendered_frame` variable is a list, and lists do not have the `ndim` attribute. `ndim` is an attribute of NumPy arrays and is used to determine the number of dimensions of an array.

The `env.render()` method is expected to return a NumPy array representing the rendered frame, but in this case, it might be returning a list, perhaps containing multiple frames or other data structures. This is probably due to differences in the `env.render()` output between different Gym versions or environment configurations.

Suggested Changes

```
import gym
import numpy as np
# used to help with visualizing in Colab
from IPython.display import display, clear_output # import clear_output
from time import sleep
import matplotlib.pyplot as plt

# Create the environment
# Use render_mode="rgb_array" to get the frames as images and then displays them using ma
# This approach should work in Jupyter Notebooks.
env = gym.make('Taxi-v3', render_mode="rgb_array")

# create a new instance of taxi, and get the initial state
# Update for gym versions >= 0.26
```

Enter a prompt here

0 / 2000

Code Review #3 – Bullet Real-Time Physics Simulator

1

PyBullet in Colab:

https://pybullet.org/wordpress/index.php/2021/04/15/pybullet-in-a-colab/

PyBullet_Colab_Test.ipynb Cannot save changes

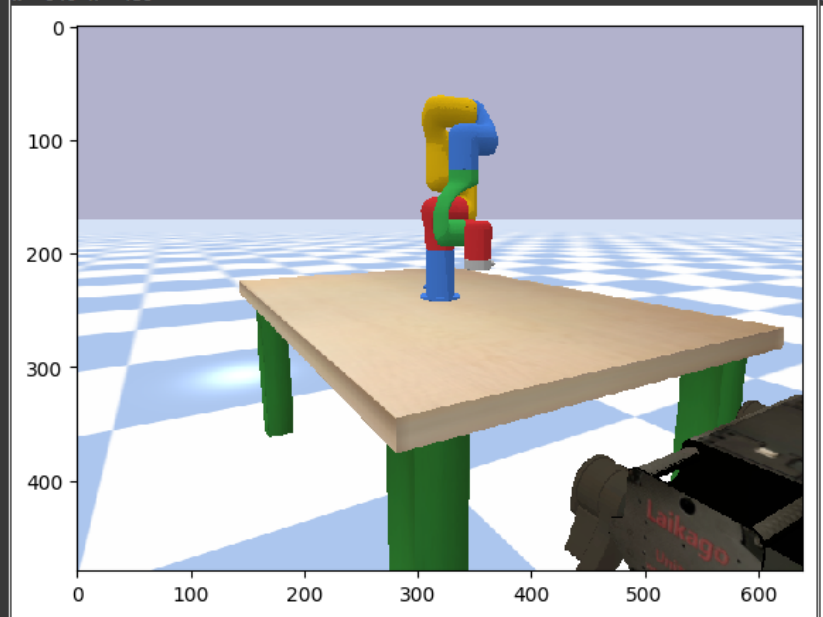
File Edit View Insert Runtime Tools Help

Commands + Code + Text Copy

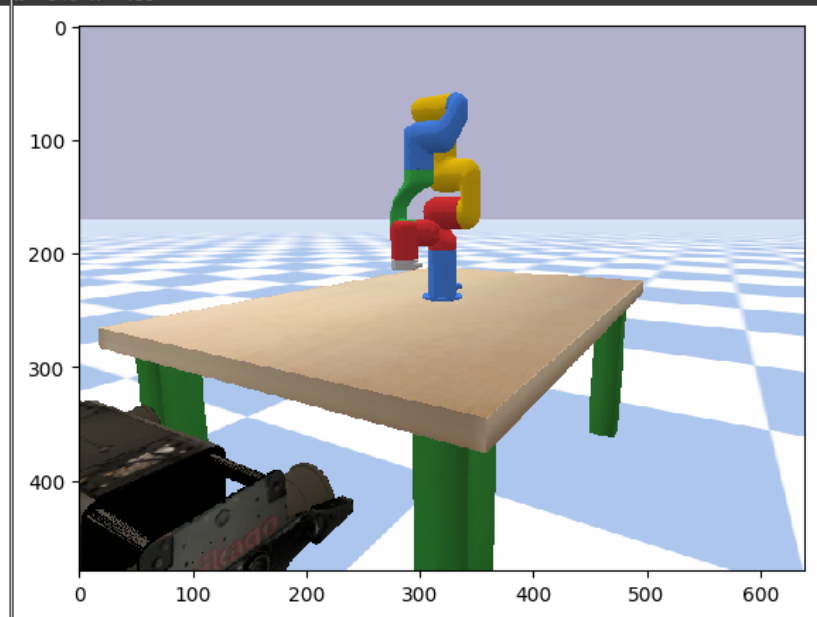
[5]

```
viewMatrix = p.computeViewMatrixFromYawPitchRoll(camTargetPos, camDistance, yaw, pitch,
                                                 roll, upAxisIndex)
aspect = pixelWidth / pixelHeight
projectionMatrix = p.computeProjectionMatrixFOV(fov, aspect, nearPlane, farPlane)

img_arr = p.getCameraImage(pixelWidth,pixelHeight,viewMatrix,projectionMatrix)
w = img_arr[0]  #width of the image, in pixels
h = img_arr[1]  #height of the image, in pixels
rgb = img_arr[2]  #color data RGB
dep = img_arr[3]  #depth data
print("w=",w,"h=",h)
np_img_arr = np.reshape(rgb, (h, w, 4))
np_img_arr = np_img_arr * (1. / 255.)
pylab.imshow(np_img_arr, interpolation='none', animated=True, label="pybullet")
```

```
w= 640 h= 480
```




```
w= 640 h= 480
```



Resources

Source:
https://youtu.be/GVQUGNv33LY?si=eh7ODoHd1yMAzyfr

2.1 A k-armed Bandit Problem

Consider the following learning problem. You are faced repeatedly with a choice among k different options, or actions. After each choice you receive a numerical reward chosen from a stationary probability distribution that depends on the action you selected. Your

objective is to maximize the expected total reward over some time period, for example, over 1000 action selections, or *time steps*.

This is the original form of the *k-armed bandit problem*, so named by analogy to a slot machine, or "one-armed bandit," except that it has k levers instead of one. Each action selection is like a play of one of the slot machine's levers, and the rewards are the payoffs for hitting the jackpot. Through repeated action selections you are to maximize your winnings by concentrating your actions on the best levers. Another analogy is that of a doctor choosing between experimental treatments for a series of seriously ill patients.

Source:
Sutton, R. S., & Barto, A. G. (2020). Reinforcement learning: An introduction (2nd ed.). The MIT Press